

30. Le problème de la documentation

Imaginez une API avec :

```
50 endpoints
```

Comment documenter tout cela ?

On pourrait écrire un document à la main :

```
GET /players

Description :
Récupère la liste des joueurs.

Paramètres :
...

Réponse :
...
```

Mais cela présente un problème :

Le code et la documentation peuvent finir par ne plus correspondre.

FastAPI propose une autre approche.

31. OpenAPI

OpenAPI est une spécification permettant de décrire une API.

Elle permet notamment de décrire :

- les routes ;
- les méthodes HTTP ;
- les paramètres ;
- les modèles ;
- les données d'entrée ;
- les données de sortie ;
- les réponses possibles.

FastAPI génère automatiquement cette description.

32. OpenAPI JSON

Notre serveur expose automatiquement :

```
/openapi.json
```

Par exemple :

```
http://127.0.0.1:8000/openapi.json
```

Nous obtenons un document JSON décrivant notre API.

Il contient notamment :

```
paths
components
schemas
parameters
responses
...
```

Ce document constitue le **contrat machine-readable** de notre API.

33. De FastAPI à OpenAPI

Voici le mécanisme fondamental à retenir :

Code Python

Type Hints

FastAPI

Pydantic

OpenAPI JSON

Swagger UI
/docs

ReDoc
/redoc

34. Swagger UI

FastAPI génère automatiquement une interface interactive :

`/docs`

Par exemple :

`http://127.0.0.1:8000/docs`

Cette interface est appelée :

Swagger UI

Elle permet de :

- voir les endpoints ;
- voir les paramètres ;
- voir les modèles ;

- tester les requêtes ;
 - observer les réponses.
-

35. Tester une API avec Swagger

Dans Swagger UI :

```
GET /players
```

Puis :

Try it out

et :

Execute

Swagger envoie réellement la requête à notre serveur.

Nous obtenons par exemple :

```
Request URL:  
http://127.0.0.1:8000/players  
  
Response code:  
200
```

Puis :

```
[  
  {  
    "id": 1,  
    "name": "Alice",  
    "score": 1200,  
    "level": 12  
  }  
]
```

36. Swagger n'est pas l'API

Attention à une distinction importante.



OpenAPI décrit l'API.

Swagger UI est une interface permettant de visualiser et tester cette description.

37. ReDoc

FastAPI fournit également une autre interface :

`/redoc`

Par exemple :

`http://127.0.0.1:8000/redoc`

Nous avons donc :

URL	Rôle
<code>/docs</code>	Swagger UI
<code>/redoc</code>	ReDoc
<code>/openapi.json</code>	Contrat OpenAPI

38. Pourquoi cette génération automatique est intéressante ?

Nous avons écrit :

```
class Player(BaseModel):  
    name: str = Field(min_length=3)  
    score: int = Field(ge=0)  
    level: int = Field(ge=1)
```

Et :

```
@app.post("/players")  
def create_player(player: Player):  
    return player
```

FastAPI peut en déduire :

```
POST /players  
  
Body:  
Player  
  
name → string, minimum 3 caractères  
score → integer, minimum 0  
level → integer, minimum 1
```

Nous n'avons pas écrit cette documentation manuellement.

39. Le rôle des Type Hints

C'est probablement l'une des idées les plus importantes de FastAPI.

Dans FastAPI, les Type Hints ne servent pas uniquement à rendre le code plus lisible.

Ils permettent au framework de comprendre notre API.

Exemple :

```
@app.get("/players/{player_id}")
def get_player(player_id: int):
    ...
```

FastAPI comprend :

```
player_id

integer
```

Et :

```
def create_player(player: Player):
```

lui indique :

```
request body

Player
```

40. Entrées et sorties

Une API possède deux directions de données.



Pour les données reçues :

```
def create_player(player: Player):
```

Pour les données retournées :

```
def get_player(...) -> Player:
```

ou explicitement :

```
@app.get(
    "/players/{player_id}",
    response_model=Player
)
def get_player(player_id: int):
    ...
```

41. Pourquoi définir un modèle de réponse ?

Le modèle de réponse permet de définir ce que l'API doit retourner.

Par exemple :

```
class Player(BaseModel):
    id: int
    name: str
    score: int
    level: int
```

Puis :

```
@app.get("/players/{player_id}", response_model=Player)
def get_player(player_id: int):
    ...
```

Nous avons maintenant un contrat explicite :

```
GET /players/{player_id}
```

Response:

```
{
  "id": integer,
  "name": string,
  "score": integer,
  "level": integer
}
```

42. Un modèle pour la création, un autre pour la réponse

Dans une véritable application, nous ne voulons pas forcément utiliser le même modèle partout.

Par exemple :

```
class PlayerCreate(BaseModel):
    name: str
    score: int
    level: int
```

Et :

```
class Player(BaseModel):
    id: int
    name: str
    score: int
    level: int
```

Pourquoi ?

Parce que l'id est généré par le serveur.

Le client ne doit donc pas nécessairement l'envoyer.

Client	Server
name	
score	
level	
	id
	généré par le serveur

43. Notre API complète

Nous pouvons maintenant imaginer notre première version :

```
from fastapi import FastAPI
from pydantic import BaseModel, Field

app = FastAPI()

class PlayerCreate(BaseModel):
    name: str = Field(min_length=3)
    score: int = Field(ge=0)
    level: int = Field(ge=1)

class Player(PlayerCreate):
    id: int

@app.get("/players")
def get_players():
    ...
```

```

@app.get("/players/{player_id}")
def get_player(player_id: int):
    ...

@app.post("/players")
def create_player(player: PlayerCreate):
    ...

@app.put("/players/{player_id}")
def update_player(
    player_id: int,
    player: PlayerCreate,
):
    ...

@app.delete("/players/{player_id}")
def delete_player(player_id: int):
    ...

```

Nous avons déjà la structure d'une véritable API REST.

44. Architecture conceptuelle

À ce stade, il faut avoir compris cette chaîne :



Code Python

Données

JSON

Et parallèlement :

FastAPI

Type Hints

Pydantic

OpenAPI

Swagger UI
/docs

ReDoc
/redoc

45. Défi — concevoir une API

À vous de jouer

Notre jeu possède maintenant des **armes**.

Une arme possède :

```
id
name
damage
weight
```

Proposez les endpoints nécessaires pour :

1. récupérer toutes les armes ;
2. récupérer une arme ;
3. créer une arme ;
4. modifier une arme ;
5. supprimer une arme.

Essayez de respecter les conventions REST.

46. Correction

Une solution possible :

```
GET    /weapons
GET    /weapons/{weapon_id}

POST   /weapons

PUT    /weapons/{weapon_id}

DELETE /weapons/{weapon_id}
```

On retrouve :

```
GET    → lecture
POST   → création
PUT    → modification
DELETE → suppression
```

47. Défi — définir le modèle Pydantic

Nous voulons qu'une arme respecte les règles suivantes :

```
name    → au moins 3 caractères
damage  → entier positif
weight  → nombre positif
```

Comment définir le modèle ?

48. Correction

```
from pydantic import BaseModel, Field

class WeaponCreate(BaseModel):
    name: str = Field(min_length=3)
    damage: int = Field(gt=0)
    weight: float = Field(gt=0)
```

FastAPI peut maintenant utiliser ce modèle pour :

- valider les requêtes ;
 - générer le schéma OpenAPI ;
 - documenter automatiquement les champs.
-

49. Défi — trouver l'information

Notre API est lancée.

Vous savez qu'elle possède :

```
GET /players
POST /players
GET /weapons
POST /weapons
```

Mais vous ne connaissez pas tous les détails.

Question

Comment découvrir automatiquement comment utiliser cette API ?

50. Réponse

Avec Swagger UI :

```
/docs
```

ou avec ReDoc :

```
/redoc
```

Et pour obtenir directement le contrat machine-readable :

```
/openapi.json
```

51. Ce qu'il faut retenir

API

Une API permet à des applications de communiquer entre elles.

```
Client ← HTTP → Serveur
```

REST

Une API REST organise généralement les opérations autour de ressources :

```
GET
POST
PUT
DELETE
```

FastAPI

FastAPI permet de créer rapidement des APIs Python.

```
@app.get("/players")
def get_players():
    ...
```

Pydantic

Pydantic permet de définir et valider les données :

```
class Player(BaseModel):
    name: str
    score: int
```

Type Hints

FastAPI exploite les annotations de type :

```
player_id: int
```

OpenAPI

OpenAPI décrit automatiquement notre API.

```
/openapi.json
```

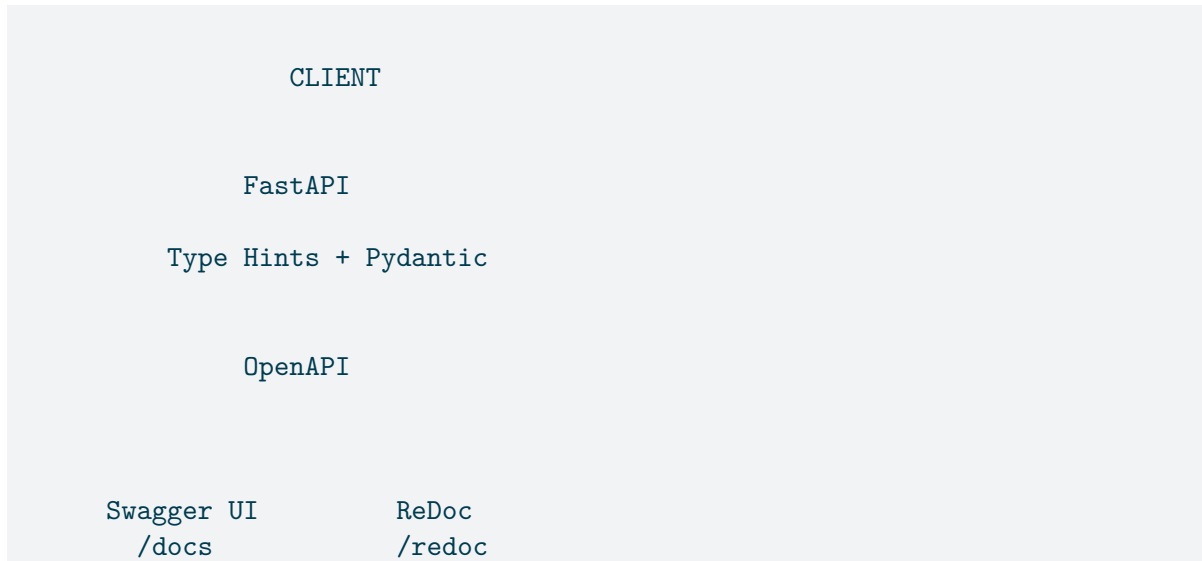
Swagger UI

Swagger UI fournit une interface interactive :

```
/docs
```

52. Le schéma à retenir





53. Pour le projet fil rouge

Pour la première partie de votre projet, vous devrez être capables de :

1. Créer une application FastAPI

```
from fastapi import FastAPI

app = FastAPI()
```

2. Définir des routes

```
@app.get("/")
@app.post("/")
@app.put("/")
@app.delete("/")
```

3. Utiliser les Type Hints

```
player_id: int
```

4. Créer des modèles Pydantic

```
class Player(BaseModel):  
    ...
```

5. Valider les données

```
score: int = Field(ge=0)
```

6. Définir les réponses

```
response_model=Player
```

7. Vérifier la documentation

```
/docs  
/redoc  
/openapi.json
```

54. La philosophie FastAPI

Une dernière idée à retenir :

Décrire correctement son API dans le code permet à FastAPI de faire beaucoup de travail automatiquement.

Code Python

+

Type Hints

+

Pydantic

FastAPI

Validation

Sérialisation

Gestion des erreurs

OpenAPI

Swagger UI